

國立臺灣科技大學資訊工程系

台巴學生

114 學年度第 2 學期專題研究

總報告

**DNS-Level Connection Interception, Analysis,
and Maliciousness Scoring System**

研究組員

Sebastián Pérez

F11315118

Francisco Vega

F11315122

指導教授： _____

中華民國 2026 年 06 月 06 日

AegisDNS - DNS-Level Connection Interception, Analysis, and Maliciousness Scoring System

1. Abstract

AegisDNS is a security application designed to help users identify and respond to web-based threats in real time. The system consists of three main components: a PySide6 desktop application with Docker functionality, a browser extension compatible with browsers built around the Manifest V3 API, and a local Flask server backend acting as the communication bridge between them. The desktop application contains a heuristic scoring scanner engine which evaluates the addresses and domains the user comes across and produces a verdict based on three main signal categories: WHOIS metadata, DNS configuration, and HTTP level settings and headers. In addition, the user can optionally employ a locally-hosted large language model (Llama 3.2 3B Instruct) to generate simple explanations of each scan result that do not require technical knowledge to understand. The application also contains many tools which can be useful for the user, including packet-level traffic analysis, scan result and navigation history files, as well as domain whitelisting and blacklisting, system notifications, and exportable reports. The browser extension operates in four configurable modes, ranging from just passively recording traffic to actively intercepting navigation. Lastly, the Flask server acts as the intermediary between the app and the extension, handling scan requests, navigation logging, and access control synchronization.

Keywords: DNS security, network monitoring, heuristic analysis, threat detection, large language model, real-time protection, browser extension.

2. Introduction

Web browsing is central to how people work, communicate, and access information today. However, that same prevalence has made it one of the main points of cybersecurity attacks in the modern day. Malicious websites, phishing pages, and deceptive domains are cheap to create, difficult to distinguish from legitimate ones, and are becoming continuously easier to create with new tools such as AI. Furthermore, the users that tend to be at most risk are those which are the least equipped to recognize the warning signs.

AegisDNS is a security tool built to address this gap. It monitors browser activity in the background, analyses the domains a user visits using a heuristic scoring engine, and explains the results in plain language using an LLM so that no technical knowledge is required. The level of protection the AegisDNS provides is adjustable. Users can run the tool silently in the background to build a log of visited domains, have scans run automatically without any interruption, or set the extension to intercept every navigation so it can be analyzed before loading. Domains can be whitelisted to allow connections to it without inspection, or blacklisted to block future connections entirely.

The system is composed of three parts: a PySide6 [1] desktop application that serves as the control interface and scan dashboard, a Chrome browser extension that logs and intercepts navigations in real time, and a local Flask server that bridges the extension and application together. A locally-run AI model translates raw scan results into natural-language summaries and recommendations, keeping all user data on-device.

The remainder of this report is structured as follows: Section 3 provides background on DNS infrastructure and related work. Section 4 describes the system architecture and the design of each component. Section 5 details the heuristic scoring engine and its signal categories. Section 6 covers the locally-hosted AI model and its integration. Sections 7 and 8 discuss the Docker [2] deployment and project website respectively. Lastly, the conclusion can be found in Section 9.

3. Background & Related Work

3.1. Background and Technical Foundations

3.1.1. Domain Name System (DNS)

The Domain Name System, also known as DNS, is a popular and foundational protocol from the Internet. Its main purpose is to translate human readable domain names into machine readable IP addresses. Once the user attempted to access a website, a query is sent to a recursive resolver, which purpose is to forward the query to a hierarchy of name servers from root servers, to TLD servers, to end in the authoritative name server for the target domain, this process ends once the connection is stabilized when the corresponding IP address is returned. This process is repeated for every web request a user performs, and makes the DNS an important component of everyday browsing.

3.1.2. DNS-Based Threats and Malicious Web Infrastructure

DNS has a vast variety of malicious actors, threat categories operating at the DNS level include Routing Manipulation & Redirection, Data Exfiltration & Stealth, DNS Availability Attacks (DDoS), and the construction of phishing and malware distribution networks built on deceptive domain registration. The common thread across these attack classes is that they all abuse the DNS protocol that most users and even many organizations implicitly trust and rarely scrutinize.

The main concern is the low barrier to entry for attackers, since registering a deceptive domain, configuring DNS records, and standing up a convincing replica of a legitimate website requires minimal technical expertise and negligible cost, while the potential impact on victims could range from credential theft to financial fraud.

3.1.3. Threat Scoring and Maliciousness Indicators

Assessing the safety of a domain or URL requires the aggregation and analysis of multiple independent signals, as no single indicator is sufficient to determine malicious intent with high confidence. Three primary data sources are used in threat scoring systems:

1. WHOIS: Records provide registration metadata for a domain, including the registrant's information, registration date, expiration date, and registrar. Newly registered domains, privacy masked registrants, and short registration periods are well established indicators of potentially malicious intent, as threat actors frequently register disposable domains for short time usage.
2. DNS: This metric reveals the technical part of a domain, it includes its associated IP addresses, its mail communication configuration and its structure. Reliable threats signals that this metric can measure are irregular behaviors such as rapid IP changes, association with known malicious hosting ranges, or misconfigured records.
3. Web: This metric examines the content present in the domain, this includes page structure, embedded scripts, redirect chains, and comparison of visual similarity to legitimate sites. This layer is very effective in order to correctly identify phishing pages that may otherwise present clean WHOIS and DNS profiles.

3.2. Related Work

3.2.1. VirusTotal

VirusTotal is a threat intelligence platform [3] that combines the scanning capabilities of more than 70 antivirus engines, URL scanners and domain analysis tools into a single API. The moment a URL, domain or IP address is sent to VirusTotal, it is evaluated against all the antivirus engines simultaneously and the results are compiled into a structured report indicating the number and identity of engines that identify the target as malicious. Considering its vast virus engine repertoire and its transparent and effective reporting, VirusTotal is a standard reference for both commercial security products and academic threat detection research.

3.2.2. Malwarebytes

Malwarebytes is a cybersecurity platform [4] that provides threat protection across two complementary components. Its desktop application offers malware detection, and real time protection against malicious processes, ransomware, and exploits at the operating system level. Complementing this, Malwarebytes Browser Guard is a browser extension that operates at the browsing layer, blocking access to known malicious URLs, phishing domains, scam pages, and intrusive trackers before the page content is loaded.

4. System Design

4.1. System Architecture Overview

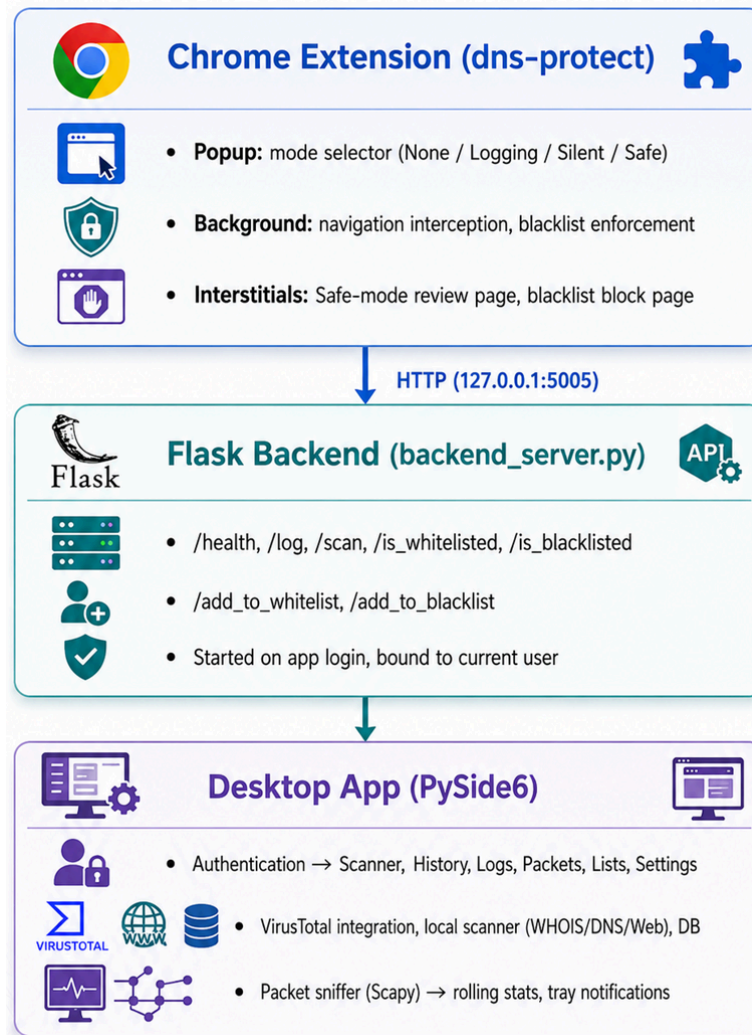


Figure 1: System architecture of AegisDNS

AegisDNS follows a three-tier client-bridge-engine architecture, illustrated in **Figure 1**. This system is split across these three components, with each being responsible for a distinct role. The browser extension handles and monitors traffic at the browser level, the desktop application performs all computation, data storage, and data management, and the Flask server acts as the communication layer between them by exposing endpoints the extension can utilize to send browsing information over to the application.

It is important to note, however, that the desktop application can function independently of the extension. The user can still choose to manually scan addresses as well as use the tools provided by the app without the web extension downloaded.

4.2. Desktop Application

The desktop application is the core foundation of AegisDNS, serving as the primary interface through which the user can interact with the entire system. It is built using PySide6, which is a library used for GUI design in Python. The user can navigate through the interface and access all of the different functionalities through a sidebar found on the left side of the window.

4.2.1. Authentication

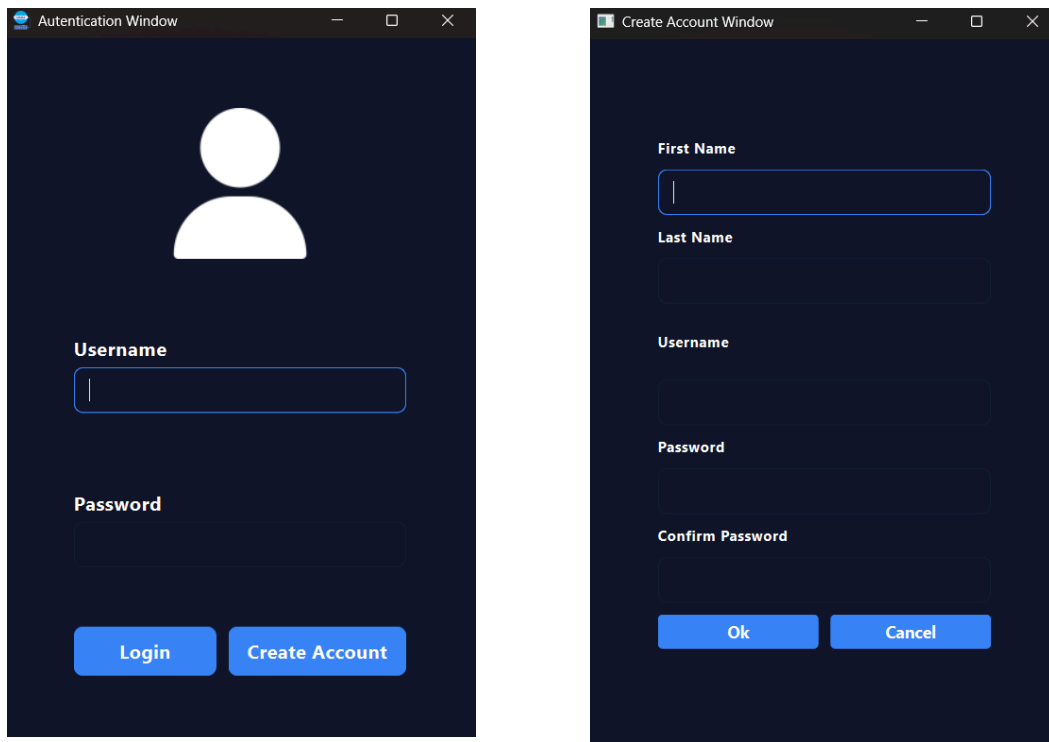


Figure 2: Login window (left) and account creation window (right)

When first running the program, the user will be met by an authentication window requesting their username and password, as seen in the left image of **Figure 2**. If the user has yet to register an account or they want to have multiple accounts, they can do so by selecting the **Create Account** button. This takes them to another window, which can be observed in the right image of **Figure 2**. In this window, the user is asked to provide their first name, last name, username, and password. If all fields are properly filled, then pressing the **Ok** button registers the user in the database and takes them back to the original authentication window, where they can log in with their account.

4.2.2. User Data Management

UserName VARCHAR	Password VARCHAR	FirstName VARCHAR	LastName VARCHAR
filter	filter	filter	filter
Nico	\$2b\$12\$0wA0NZhc8FUIB/eEQu3ryuWf0sUCNBJUAAU.JgJ	Francisco	Vega
Fabricio	\$2b\$12\$7hEngDNPQ37SKQd0uTCAwuF5SZi.yXOU3Ma1K5J	Franni	asdf

Figure 3: User table

Address VARCHAR	Date DATETIME	Verdict VARCHAR	owner VARCHAR
filter	filter	filter	filter
https://www.reddit.com/?solution=5ebded4c7859;	2026-06-04 19:54:07.414512	SAFE	Nico
https://www.instagram.com/	2026-06-04 19:54:29.961979	SAFE	Nico
https://www.facebook.com/?locale=es_LA	2026-06-04 19:54:34.901703	SAFE	Nico

Figure 4: Addresses table

The user data is stored in a SQLite database built and managed through the SQLAlchemy [5] architecture.

The user table, that can be observed in **Figure 3**, stores each account's username as its primary key, alongside the user's first name, last name, and password. An additional table is the Addresses table, visible at **Figure 4**, which records the history of scans performed by each user, storing the target address, timestamp, scan verdict and a foreign key reference to the owning user account.

In order to guarantee user's safety, passwords are not stored in plain text, they are encrypted using bcrypt [6], a powerful library that is professional standard adaptive hashing algorithm designed specifically for password storage. bcrypt incorporates a randomly generated salt per hash, ensuring that identical passwords produce distinct hash values and rendering precomputed lookup attacks ineffective. Authentication is performed by comparing the raw input against the stored hash using bcrypt's dedicated verification function, so the original password is never reconstructed or exposed at any point in the application's lifecycle.

Database operations that revolves around user creation, authentication, credential updates, username changes, and scan history logging are encapsulated within a dedicated DatabaseManager class that exposes a set of static methods. This design centralizes all data access logic, isolates session and transaction management, and ensures that database errors trigger a session rollback to preserve data integrity.

4.2.3. Scanner

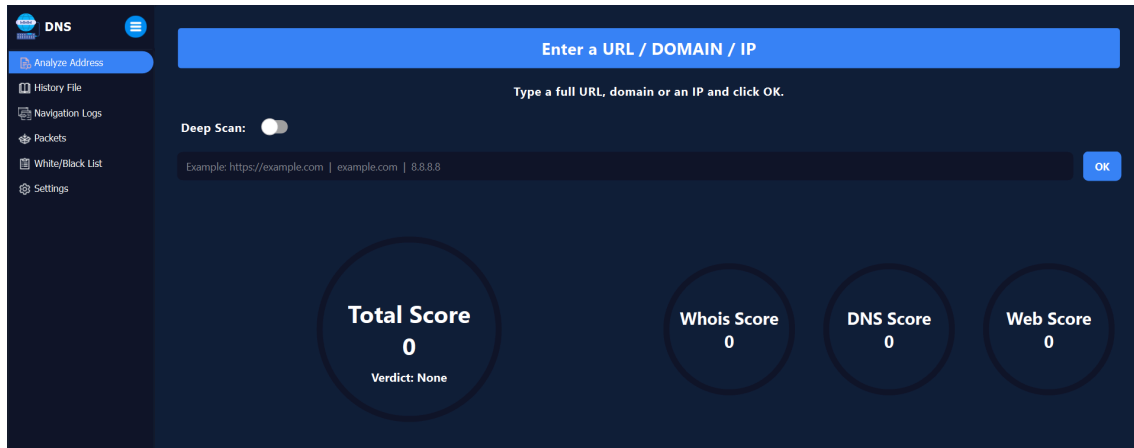


Figure 5: Scan window

The scanner window in **Figure 5** is the first interface presented to the user after logging in. The user can input any IP address or domain in the search bar to scan it and produce a verdict. The verdict output depends on the maliciousness score assigned to the address by the scanner after its analysis.

The possible verdicts range across seven levels — from SECURE to MALICIOUS — determined by a risk score produced by the scanner’s analysis. The score reflects contributions from three signal categories: WHOIS metadata, DNS configuration, and web-level settings, each of which can raise or lower the final result.

Risk Score	Verdict
0 – 10	SECURE
11 – 20	SAFE
21 – 30	NEUTRAL
31 – 40	CAUTION
41 – 50	SUSPICIOUS
51 – 59	DANGEROUS
60+	MALICIOUS

Table 1: Risk score to verdict mapping

The verdict scores mainly range from 0 to 60, with 0 being the safest score and 60 being the most dangerous. By default, any entry starts with a score of 30 (NEUTRAL). Scores greater than 60 are possible but are rare occurrences.

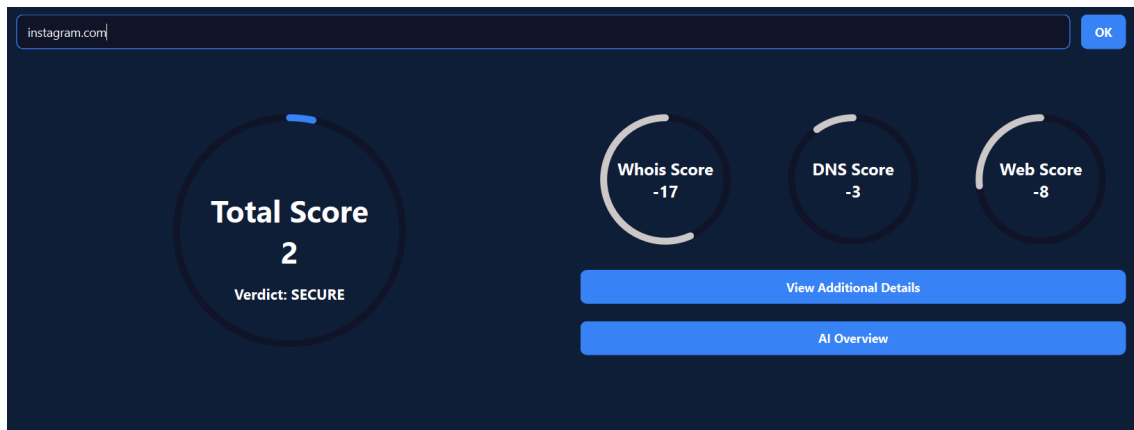


Figure 6: Instagram scan result

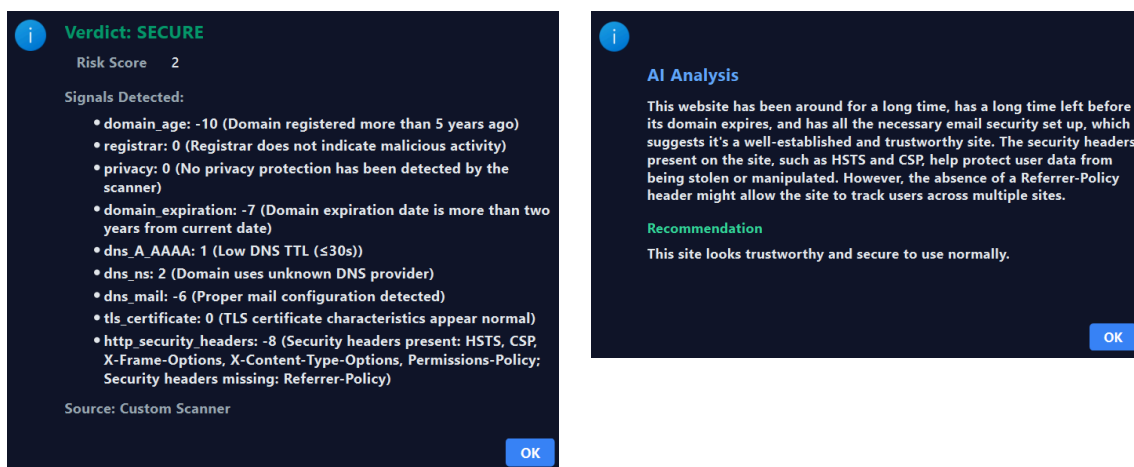


Figure 7: Additional details panel (left) and AI Overview (right)

As an example, the scan result in **Figure 6** can be observed. Instagram receives a total score of 2 and a **SECURE** verdict from the scanner, with every signal category reacting positively overall. The scores are also visually reflected by the circular bar graphs.

By pressing the **View Additional Details** button, the user may check each individual signal score along with a brief description, as seen in the left image of **Figure 7**. Additionally, the user can also press the **AI Overview** option to receive an explanation of the results in simpler terms. The LLM model also provides a brief recommended action to the user with regards to the address scanned. An example output from the LLM can be observed in the right image of **Figure 7**.

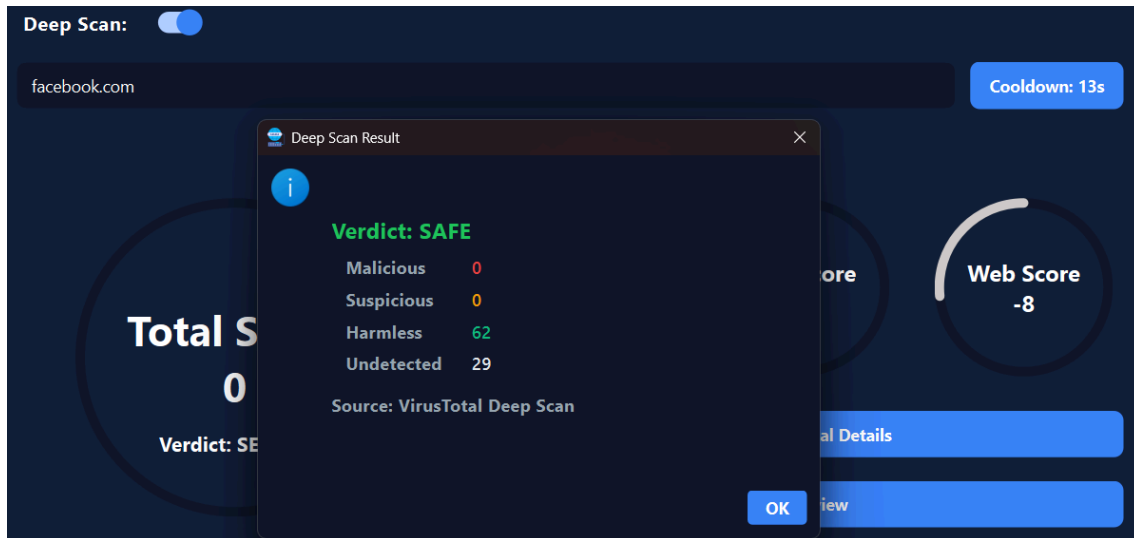


Figure 8: VirusTotal deepscan functionality

Lastly, the scanner also provides a deep scan functionality which uses the VirusTotal API for analysis. This evaluates the address with 91 antivirus engines, offering a second layer of analysis independent of the local scanner. However, the VirusTotal API is rate-limited, only allowing up to four scans per minute.

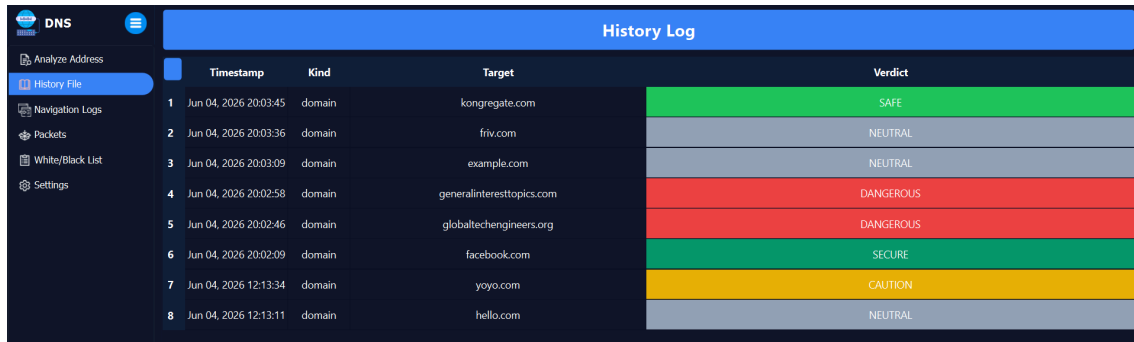
An example scan using *facebook.com* is showcased in **Figure 8**, where the scan results indicate that 62 engines gave a **harmless** verdict, and 29 returned **undetected**. It is important to note that these labels belong to VirusTotal classification system and that each engine uses its independent detection methodology.

Condition	AegisDNS Verdict
More than 3 engines flag as malicious	BLOCK
1–3 engines flag as malicious, or any flag as suspicious	CAUTION
No engines flag as malicious or suspicious	SAFE

Table 2: AegisDNS verdict mapping for VirusTotal deep scan results

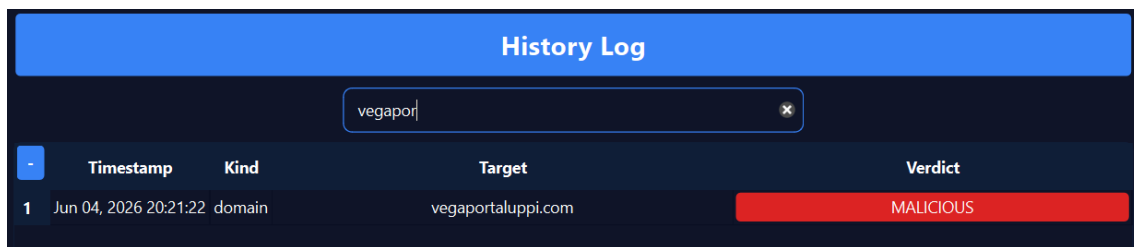
AegisDNS then maps these results to its own simplified three-level verdict system, based on the number of engines that flagged the target as malicious or suspicious. The mapping methodology is showcased in **Table 1**.

4.2.4. History File



	Timestamp	Kind	Target	Verdict
1	Jun 04, 2026 20:03:45	domain	kongregate.com	SAFE
2	Jun 04, 2026 20:03:36	domain	friv.com	NEUTRAL
3	Jun 04, 2026 20:03:09	domain	example.com	NEUTRAL
4	Jun 04, 2026 20:02:58	domain	generalinteresttopics.com	DANGEROUS
5	Jun 04, 2026 20:02:46	domain	globaltechengineers.org	DANGEROUS
6	Jun 04, 2026 20:02:09	domain	facebook.com	SECURE
7	Jun 04, 2026 12:13:34	domain	yoyo.com	CAUTION
8	Jun 04, 2026 12:13:11	domain	hello.com	NEUTRAL

Figure 9: History File Section



	Timestamp	Kind	Target	Verdict
1	Jun 04, 2026 20:21:22	domain	vegaportaluppi.com	MALICIOUS

Figure 10: Filter Search

The history file section provides users with a structured and interactive log of all previously analyzed targets. As observed in **Figure 9**, scan records are displayed in a four-column table containing the timestamp, scan type, target address, and resulting verdict for each entry. To improve readability at a glance, verdict cells possess different colors that match the threat level, ranging from green tones for safe and secure results, through yellow for suspicious or cautionary verdicts, to red variants for dangerous and malicious classifications.

The panel incorporates a per column search and filtering system that can be seen in **Figure 10**, accessible through a right click context menu on any column header. Filter inputs persist across table reloads and can be committed or cleared independently per column. Sorting by any column in ascending or descending order is also available through the same context menu.

Each row in the table exposes a context menu on right click, offering a set of actions, such as copying the target to the clipboard, viewing detailed scan statistics cached from the original analysis, adding the entry to the user's whitelist or blacklist, deleting the record, and requesting an AI overview.

The scan history data is stored in a local JSON and JSONL files. The panel also exposes a reset function that clears the scanner cache and wipes all history and request log files, restoring them to their initial empty state.

4.2.5. Navigation Logs

Navigation Logs			
-	Timestamp	Target	Action
1	Jun 04, 2026 13:36:57	https://www.facebook.com/?locale=es_LA	SCAN
2	Jun 04, 2026 19:53:32	https://www.reddit.com/	SCAN
3	Jun 04, 2026 19:53:34	https://www.facebook.com/?locale=es_LA	SCAN
4	Jun 04, 2026 19:53:39	https://www.instagram.com/	SCAN

Figure 11: Navigation Logs Section

The navigation logs section provides users with a chronological record of all network destinations intercepted by the browser extension during active logging mode. Log entries are displayed in a three-column table containing the timestamp, target address, and an direct scanner button for each record, this table can be observed in **Figure 11**.

The panel shares the same filtering and sorting infrastructure as the History File section, having a per column search inputs are accessible through a right click context menu on any column header, filter values persist across table reloads, and ascending or descending sorting is available for all columns.

In the action column in the navigation logs table, once the user clicks the scan button, the action redirects the user to the main scanner panel, populates the input field with the selected target address, and triggers a scan automatically.

Navigation log data is stored in a dedicated JSON file within the application's cache directory. A reset button positioned in the table corner clears this file, wiping the navigation log history entirely.

4.2.6. Packets

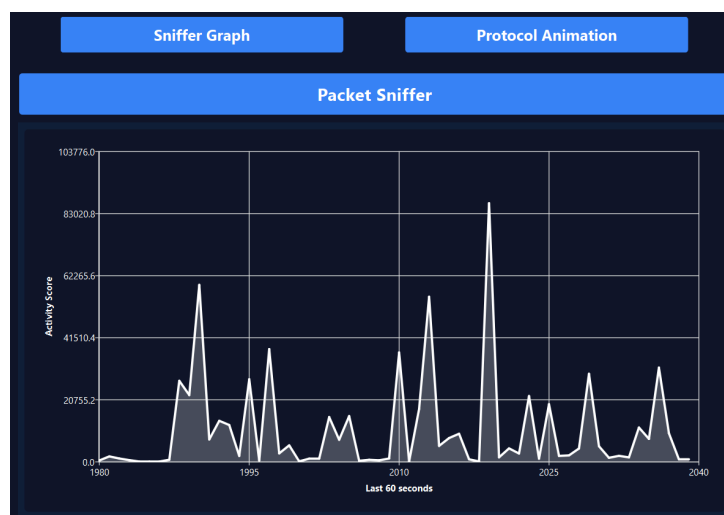


Figure 12: Packet Sniffer Graph

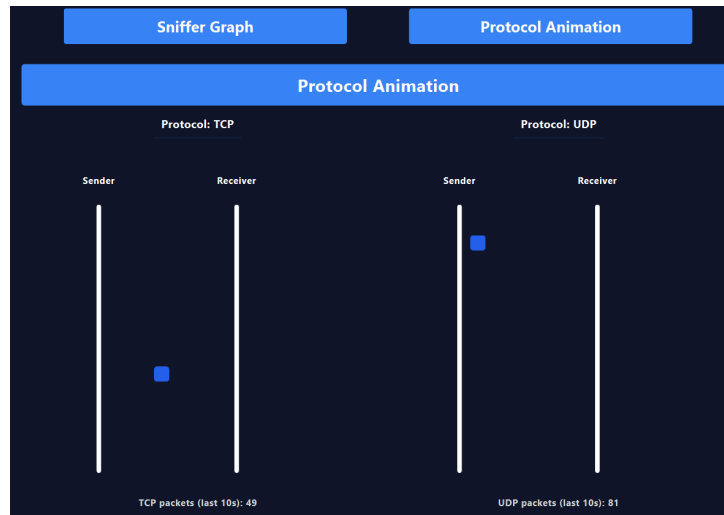


Figure 13: Protocol Animation

The packet sniffer interface, seen in **Figure 12**, provides a real-time visualization of network activity through a live-updating line chart. The graph shows the activity score across a 60 second time window, and updates every 10 seconds. The y-axis dynamically rescales based on the peak value within the visible window, ensuring that both low-traffic periods and sudden activity spikes are accurately represented without distorting the graph.

This functionality is accomplished by using Scapy [7], a Python-based packet analysis library that passively observes the flow of network packets at the interface level without intercepting, modifying, or delaying any traffic. This passive-only approach ensures that the sniffer introduces no latency or interference into the user's network activity, functioning purely as an observational layer.

The packet sniffer is a valuable feature within AegisDNS because it enables users to detect abnormal network behavior that may not be apparent through standard browsing. Unusual spikes in outbound traffic, sustained high-activity periods during idle system states, or irregular traffic patterns can all serve as indicators of background processes communicating without the user's knowledge.

The protocol animation observed in **Figure 13** panel provides a visual representation of real-time network traffic by displaying side by side animations of TCP and UDP packet flows. Each animation represents a packet traveling between the sender and a receiver, reflecting the behavior of each protocol.

The animation speed is directly tied to the packet count recorded in the most recent 10 second capture window, higher packet volumes are represented with faster animations, low packet volumes result in slower animations. This mapping between real traffic data and animation tempo gives users an intuitive sense of current network load without requiring them to interpret raw numerical data. Packet counters displayed beneath each animation show the exact number of TCP and UDP packets captured in the last 10 seconds, providing a precise reference alongside the visual representation.

4.2.7. Whitelist

White List

Black List

White List Log

	Timestamp	Kind	Target	Verdict
1	Jun 05, 2026 00:07:47	domain	globaltechengineers.org	WHITELISTED
2	Jun 05, 2026 00:07:42	domain	example.com	WHITELISTED
3	Jun 05, 2026 00:07:39	domain	reddit.com	WHITELISTED

Figure 14: Whitelist window

The whitelist window (see **Figure 14**) shows the addresses and domains the user has whitelisted along with their respective timestamp. The user can whitelist entries from the history log window as well as from the browser extension. Whitelisted addresses are exempted from getting blocked by the extension in all scenarios. The user can remove entries from the whitelist by selecting **Delete** after clicking on them.

4.2.8. Blacklist

White List

Black List

Black List Log

	Timestamp	Kind	Target	Verdict
1	Jun 04, 2026 20:31:34	domain	vegaportaluppi.com	BLACKLISTED
2	Dec 23, 2025 15:01:50	url	https://www.definitions.net/definition/asdasdasdas	BLACKLISTED
3	Dec 23, 2025 14:50:36	url	https://aniwatchtv.to/	BLACKLISTED

Figure 15: Whitelist window

The blacklist window (see **Figure 15**) shows the addresses and domains the user has blacklisted along with their respective timestamp. Like the whitelist, entries can be blacklisted from the history log window and the browser extension. If the user attempts to navigate to a blacklisted address, they are instead taken to an interstitial webpage that informs them that they are attempting to connect to a blocked address, and asks for confirmation before proceeding.

4.2.9. Settings

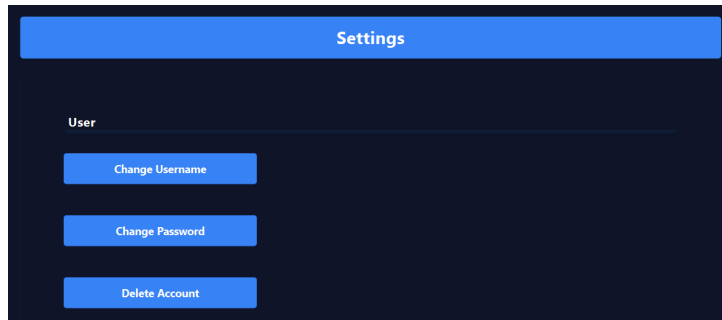


Figure 16: User Section, Settings.

The settings panel provides a centralized interface for managing user preferences and application configuration, organized into four distinct sections.

The user section, in **Figure 16**, provides the account management controls, allowing the user change their username, update their password, or permanently delete their account. Each action launches a dedicated dialog window that requires the current password for verification before any changes are committed, ensuring that account modifications cannot be performed without proper authentication.

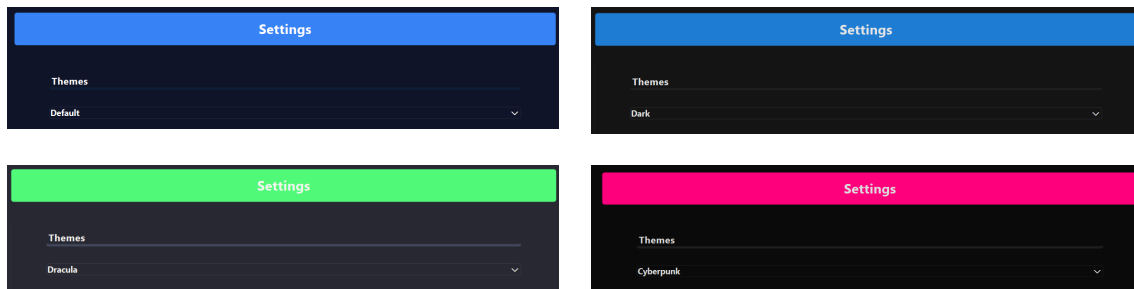


Figure 17: Themes Section, Settings.

In **Figure 17** we can observe the themes section, which has a dropdown menu offering four different themes: Default, Dark, Dracula, and Cyberpunk. Selecting a theme triggers the style update of the whole application through a centralized theme format that is stored in a JSON configuration file. The selected theme is persisted per user so that it is restored automatically on the next login.

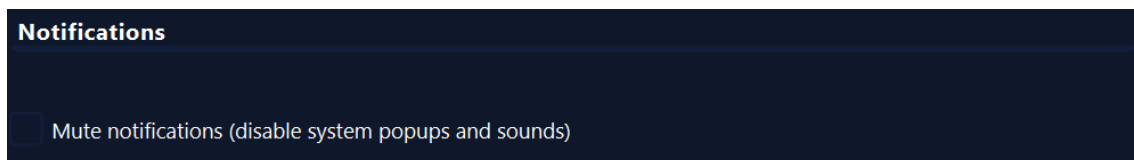


Figure 18: Notifications Section, Settings.

The Notifications section, seen in figure, contains a mute toggle that suppresses notifications popups and sound alerts generated by the application. The mute state is saved to the user's settings profile and restored across sessions.

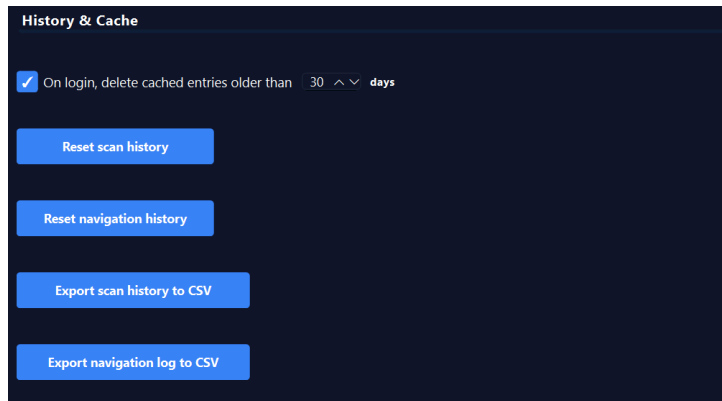


Figure 19: History & Cache Section, Settings.

The History and Cache section illustrated in figure offers controls for managing locally stored scan and navigation data. A configurable cache invalidation option automatically removes cached scan entries older than a the user defined number of days upon login, preventing stale threat intelligence from persisting indefinitely. The separation of the reset buttons allow the user to manually clear the scan history and navigation log at any time. Additionally, both the scan history and navigation log can be exported to CSV files through a file save dialog, producing structured records that include timestamps, targets, verdicts, risk scores, and scan sources that allows the users to retain or analyze their data externally.

4.3. Browser Extension

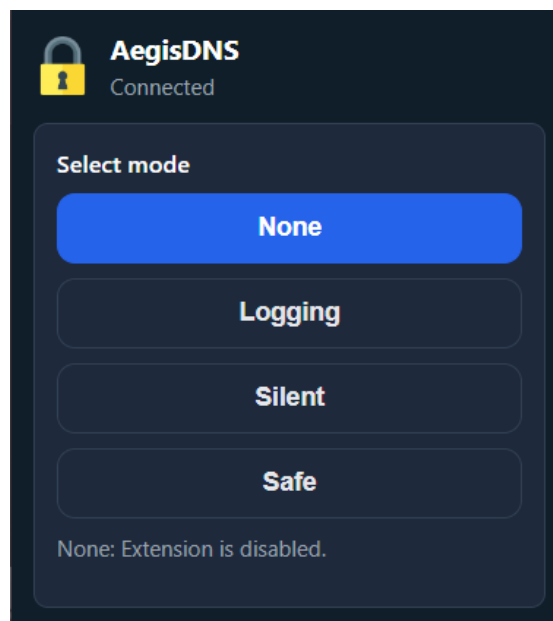


Figure 20: History & Cache Section, Settings.

The browser extension is built using HTML, CSS, and Javascript. It is compatible with any browser built on the Manifest V3 API [8], including Google Chrome and Microsoft Edge. It can operate as both a passive or active security layer sitting between the user and their browsing activity, depending on which mode is selected. The mode can be selected through the extension popup shown in **Figure 20**.

The browser extension actively communicates with the Flask server to confirm that the connection with the application is still established, log navigations, query the whitelist and blacklist, and trigger scans. It is worth noting that regardless of the active mode, navigating to a blacklisted address will always trigger the blacklist interstitial warning described in Section 4.2.8.

4.3.1. None

If the **None** mode is selected, the extension provides no functionality. However, the `/health` Flask endpoint is still continuously queried to check if the connection with the desktop application remains ongoing.

4.3.2. Logging Mode

If **Logging** mode is active, the extension passively records every navigation the user performs, sending each address to the Flask server via a POST request to the `/log` endpoint. No interception or automatic scanning occurs. The user can see the resulting logs in the **Navigation Logs** section of the desktop application, and can choose to scan the address from there.

4.3.3. Silent Mode

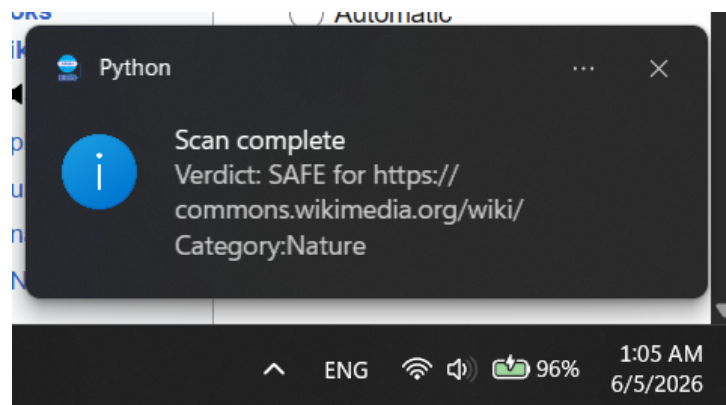


Figure 21: Notification

The **Silent** mode extends **Logging** mode by additionally triggering an automatic scan for each address visited through the `/scan` Flask endpoint. No interception occurs, but every single navigation is now evaluated in the background. A notification is sent each time a scan is completed (unless disabled in the settings), as shown in **Figure 21**, which provides both the verdict and address scanned. This gives the user a seamless browsing experience while also having real-time continuous security monitoring.

4.3.4. Safe

Safe mode provides the highest level of protection for the user. While active, any navigation that is detected is intercepted, and the extension immediately redirects the user to an interstitial while the Flask server is queried to check if the address is one of the existing whitelist and blacklist entries. Depending on the result, three actions can occur:

- Whitelisted entry

If the address is found in the whitelist, the interstitial is skipped and the user can access the site directly.

- Blacklisted entry

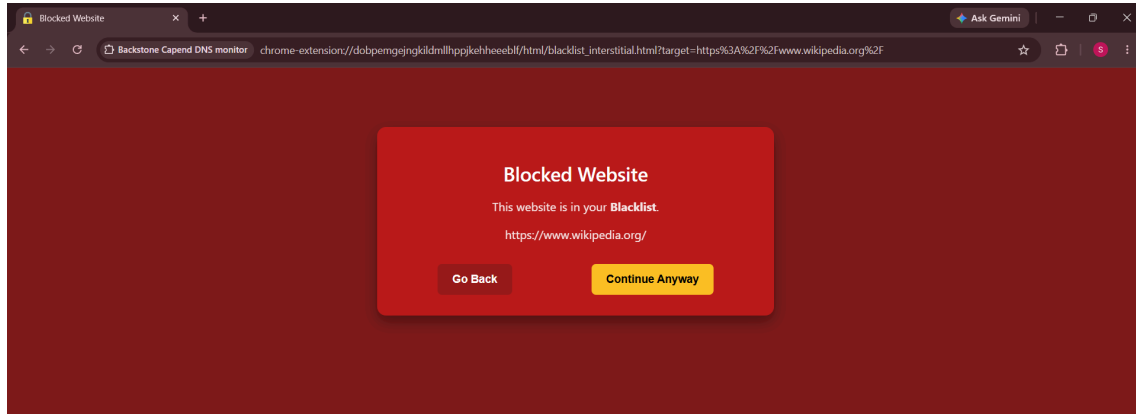


Figure 22: Blacklist interstitial

If the address is found in the blacklist, then the user is taken to a special interstitial that informs them that they are trying to access a blocked address, and allows them to choose whether to approve the connection or not. This interstitial can be observed above in **Figure 22**.

- New entry

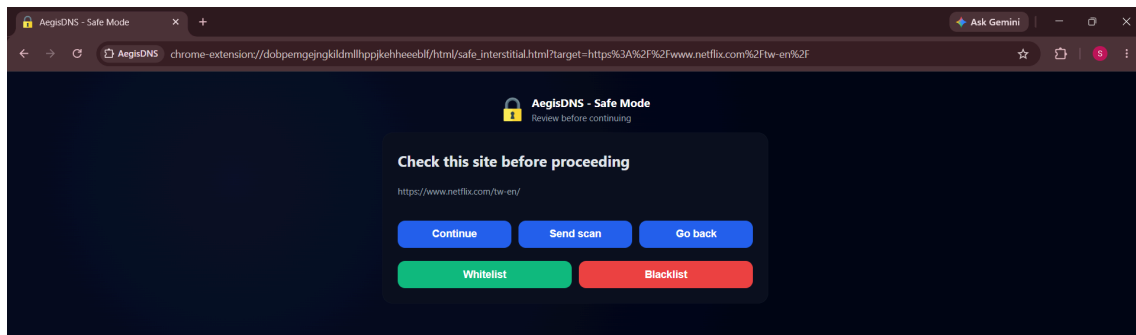


Figure 23: Default interstitial

If the address is unknown, the user is taken to the default interstitial shown in **Figure 23**, where they are given five choices: continue to the site, request a scan, return to the previous site, whitelist, and blacklist.

Safe mode is particularly useful when visiting suspicious websites which attempt to constantly redirect you to other sites. It also pairs well with the whitelist, since as the user accumulates trusted entries over time, the number of interstitials and interceptions they encounter naturally begins decreasing. This makes the mode progressively less intrusive with time while still maintaining the security benefits.

4.4. Flask Server

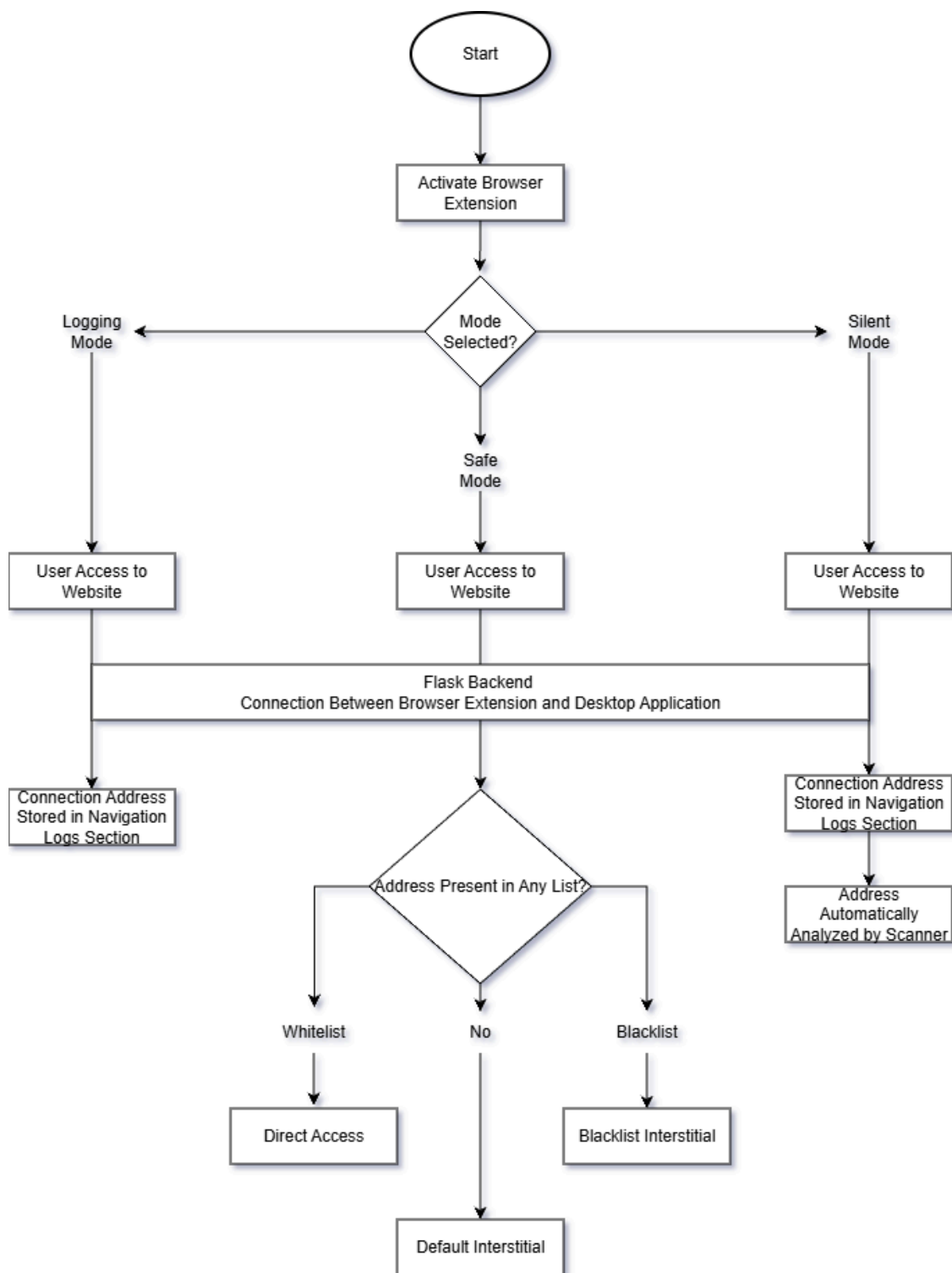
The Flask server acts as the messenger between the browser extension and the desktop application. The server is started automatically the moment the user logs into their

account, and remains active for as long as the session is ongoing. It listens locally on port 5005, exposing a REST API over HTTP. All endpoints are restricted to localhost, meaning that no external network access is required. All of the endpoints exposed by the Flask server can be viewed in **Table 3**.

Endpoint	Method	Description
/health	GET	Returns server status, current mode, and bound username. Used by the extension to verify connectivity every 30s.
/log	POST	Receives a navigation event from the extension and appends it to the navigation log file.
/scan	POST	Accepts a URL or domain and triggers a background scanner thread to analyze it.
/is_whitelisted	POST	Checks whether a given URL matches any entry in the current user's whitelist.
/is_blacklisted	POST	Checks whether a given URL matches any entry in the current user's blacklist.
/add_to_whitelist	POST	Adds a URL to the current user's whitelist from the extension.
/add_to_blacklist	POST	Adds a URL to the current user's blacklist from the extension.

Table 3: Flask server REST endpoints

4.5. Flowchart



5. Scan Heuristics

The AegisDNS scanner evaluates submitted addresses by accumulating a risk score across multiple signals. The score begins at a baseline of 30, which represents a neutral address. This score is adjusted upward or downward depending on what each signal determines. Scores are clamped to the range [0,100], but it is unusual to see scores beyond the [0, 60] range, which is why verdicts seen in **Table 1** only cover that threshold.

The signals are grouped into three main categories: WHOIS, DNS, and Web. However, note that if the user comes across a raw IP address while browsing rather than a domain, the scanner bypasses these categories and evaluates the IP indicator signal instead.

5.1. IP Indicator

If a raw IP address is encountered or input to the scanner, the scanner assigns a penalty of +30 to the baseline. The main rationale for this is that coming across a raw IP address while browsing is inherently suspicious, since most legitimate websites are accessed through domain names.

IP addresses offer no verifiable identity, no certificate authority, or any validation to a named entity. Because of this, they cannot be classified by the three main signal categories, since they do not have WHOIS records, DNS configurations, or HTTP headers like domains do.

Condition	Score	Reason
Private, loopback, link-local, or reserved IP	+20	Suspicious or non-routable address space
Public routable IP	+0	Normal public address
Non-standard port (not 80 or 443)	+10	Unusual port for web traffic

Table 4: IP indicator scoring

However, the scanner still tries to infer at least some information from the IP address, which can be observed in **Table 4**. The IP type is classified as publicly routable or as a reserved address space. Private, loopback, link-local, or reserved IP addresses are not intended to be accessible over the public internet, and encountering one indicates malicious redirection or local network exploitation, which is why a +20 penalty is given. Furthermore, if a non-standard port is detected in the URL (which is anything other than port 80 for HTTP and port 443 for HTTPS), a further +10 is applied, as legitimate websites rarely expose themselves on unconventional ports.

5.2. WHOIS Signals

WHOIS signals examine the domain registration metadata retrieved from the WHOIS database.

5.2.1. Domain Age

How long a domain has been active is one of the strongest individual signals as to whether a website is legitimate or not. Threat actors tend to register disposable domains for short-term usage, as they usually get taken down early in their lifespan.

As such, domains registered recently are punished by the scanner, and domains that have been around for long periods of time are rewarded. The specific scores can be seen in **Table 5**.

Age	Score	Reason
< 7 days	+30	Domain registered less than 7 days ago
7 – 30 days	+15	Domain registered less than a month ago
30 – 90 days	+10	Domain registered less than 3 months ago
90 – 365 days	+5	Domain registered less than 1 year ago
1 – 2 years	0	Age does not indicate any particular risk
2 – 5 years	–5	Domain registered more than 2 years ago
≥ 5 years	–10	Domain registered more than 5 years ago

Table 5: Domain age scoring

5.2.2. Registrar

The registrar through which a domain is registered is checked with a list of providers known to have elevated abuse rates. This information is sourced from the Spamhaus Registrar Reputation Statistics database [9]. The specific scoring mechanism can be observed in **Table 6**.

Risk Level	Score	Examples
High	+5	NiceNIC, Aceville, NameMart
Medium	+3	REG.RU, PDR Ltd, eNom, Openprovider
Low	+1	NameSilo, Dynadot, Tucows
No match	0	Registrar does not indicate malicious activity

Table 6: Registrar risk scoring

5.2.3. Privacy Protection

The presence of WHOIS privacy protection is a minor risk indicator. While privacy services are commonly used by legitimate domain owners, they are commonly employed by threat actors to hide registrant identity. Privacy protection is detected by performing a keyword search on the raw WHOIS output. These keywords include strings such as "Redacted for Privacy" and WhoisGuard Protected, which are commonly inserted by privacy providers.

- Privacy protection detected: **+3**
- No privacy protection: **0**

5.2.4. Domain Expiration

Domains registered for short periods are more likely to be disposable. On the other hand, a domain with a long remaining registration period suggests a long-term legitimate presence. The scoring criteria can be seen in **Table 7**.



Time Until Expiry	Score	Reason
< 30 days	+5	Domain expiring imminently
30 – 90 days	+3	Domain expires within 3 months
90 – 365 days	0	Domain expires within the year
1 – 2 years	-3	Domain expiration more than 1 year away
≥ 2 years	-7	Domain expiration more than 2 years away

Table 7: Domain expiration scoring

5.3. DNS Signals

DNS signals examine the technical configuration of the domain as published in the Domain Name System.

5.3.1. A/AAAA Records and Fast-Flux Detection

The number of IP addresses a domain resolves to and the TTL (Time to Live) of those records are evaluated for fast-flux [10] behavior. Fast-flux is a technique used by malicious actors to quickly rotate between the IP addresses associated with a domain in order to evade takedowns. In order to avoid flagging large and popular domains which may have legitimate reasons for having many IP addresses and lower TTL, only extreme cases are evaluated highly, as seen in **Table 8**.

Condition	Score	Reason
$TTL \leq 30$ seconds	+1	Unusually low TTL
≥ 7 A/AAAA records	+1	High record count
≥ 10 A/AAAA records	+2	Very high record count
$TTL \leq 30s$ and ≥ 15 records	+20	Extreme fast-flux behavior detected
$TTL \leq 60s$ and ≥ 10 records	+8	Possible fast-flux behavior detected

Table 8: A/AAAA record and fast-flux scoring

5.3.2. Nameserver Provider

The DNS provider responsible for a domain is classified against a list of well known DNS providers [11]. Malicious domains are more likely to use less reputable providers.

- Unknown provider: +2
- Known reputable provider: 0

5.3.3. Mail Configuration (MX / SPF / DMARC)

Proper mail configuration is standard in most legitimately operated domains. Missing MX records, SPF policy, and a DMARC policy [12] is a moderate risk indicator. The scoring applied to each condition is shown in Table 9.



Condition	Score	Reason
No MX records	+2	No mail server configured
No SPF record	+1	No sender policy found
No DMARC record	+1	No DMARC policy found
All three missing	+2	Additional penalty: no mail configuration at all
All three present	-6	Proper mail configuration detected

Table 9: Mail configuration scoring

5.4. Web Signals

Web signals examine the properties of the server and content served over HTTP/HTTPS.

5.4.1. TLS Certificate

The presence and characteristics of a TLS certificate are evaluated. A missing certificate is the strongest single web-level risk signal, as it indicates the site operates over unencrypted HTTP. A certificate from a free certificate authority is more likely to be used by malicious actors, as they are less likely to invest on a website that will be taken down eventually.

- No TLS certificate (HTTP only): **+15**
- Certificate is from a free certificate authority (e.g., Let's Encrypt, ZeroSSL, etc): **+3**

5.4.2. HTTP Security Headers

The presence or absence of standard HTTP security headers [13] is evaluated. Well-configured servers implement these headers to protect their users, and their absence is a sign of poor security. The scoring mechanism can be observed in **Table 10**.

Header	Present	Missing
Strict-Transport-Security (HSTS)	-2	+2
Content-Security-Policy (CSP)	-2	+2
X-Frame-Options	-1	+1
X-Content-Type-Options	-1	+1
Referrer-Policy	-1	+1
Permissions-Policy	-1	+1
Bonus: 3 or more headers present	-1	—

Table 10: HTTP security headers scoring

6. AI Model

AegisDNS integrates a local large language model to translate scanner output into plain-language explanations accessible to non-technical users. Rather than presenting a raw risk score and signal list, the application passes the scan results to the model, which produces a short natural-language summary alongside a recommended action for the user.

6.1. Model Selection

The model used is Meta’s Llama 3.2 3B Instruct [14], served locally via `llama-cpp-python` [15]. The model is downloaded on first use from Hugging Face [16] directly into the user’s application data directory, and all inference runs on the local machine. This model was chosen to keep the download size (2 GB) manageable while still producing relevant and useful explanations.

6.2. Prompting Strategy

The model is provided a system prompt defining its role and output format. The constraints are as follows: explanations must be written in plain prose, must not exceed five sentences, must use hedged language rather than absolute safety claims, and must end with a single sentence prefixed **RECOMMENDATION:**, followed by a recommended action. The **RECOMMENDATION:** marker is used to split the output into two sections: explanation and recommendation, which are displayed separately in the UI.

To make the model more consistent, demonstrate the expected tone, and have a higher output quality, two few-shot examples are provided to the model. One is from a **DANGEROUS** verdict, and another is from a **SAFE** verdict. Below is the few-shot example for a **DANGEROUS** verdict.

User	Verdict: DANGEROUS Risk Score: 55 Signals: • <code>tls_certificate</code>: +15 (No HTTPS certificate; site served over HTTP only) • <code>dns_mail</code>: +6 (No MX records; No SPF record; No DMARC policy) • <code>privacy</code>: +3 (Privacy protection detected on WHOIS)
Model	This website does not use HTTPS, meaning anything you type — passwords, personal details, payment info — travels in plain unencrypted text that anyone on the same network could intercept. It also has no email infrastructure set up whatsoever, which every legitimate business has, suggesting nobody professionally operates this site. The owner’s identity is also deliberately hidden. RECOMMENDATION: Avoid entering any personal information on this site, and think twice before visiting it until you can verify who runs it.

Listing 1: Few-shot example injected into the model context (DANGEROUS verdict)

Note that signals with an absolute score below ± 3 are filtered out so that the model focuses only on the most impactful findings.

7. Docker

To ensure a consistent and reproducible deployment environment across all platforms, AegisDNS is containerized using Docker. The docker configuration englobes all application dependencies, such as python runtime, PySide6 (Qt), Scapy, Flask and the SQLAlchemy database layer into a standardized system, eliminating environment-specific compatibility issues that may arise from differences in operating system configurations or installed package versions, this guarantees that the application behaves identically whether it is deployed on Windows, macOS, or Linux.

Docker Compose defines and manages the multi-container setup through a single docker compose.yml file, allowing the Flask backend server and the desktop application to be declared as separate services that start, communicate, and shut down together as a unified environment. This means the entire application stack can be brought up with a single command, removing the need for the user to manually configure a Python environment, resolve dependency conflicts, or start individual components in the correct order.

For network monitoring functionality, the container running the packet sniffer is granted the necessary host network permissions and Linux capabilities to allow Scapy to perform packet capture at the interface level. It is worth to mentioned that the browser extension, by nature, runs directly on the host machine's browser and connects to the Flask backend container via localhost, meaning the containerization boundary encompasses the desktop application and backend while the extension remains as a host side component.

8. Landing Page

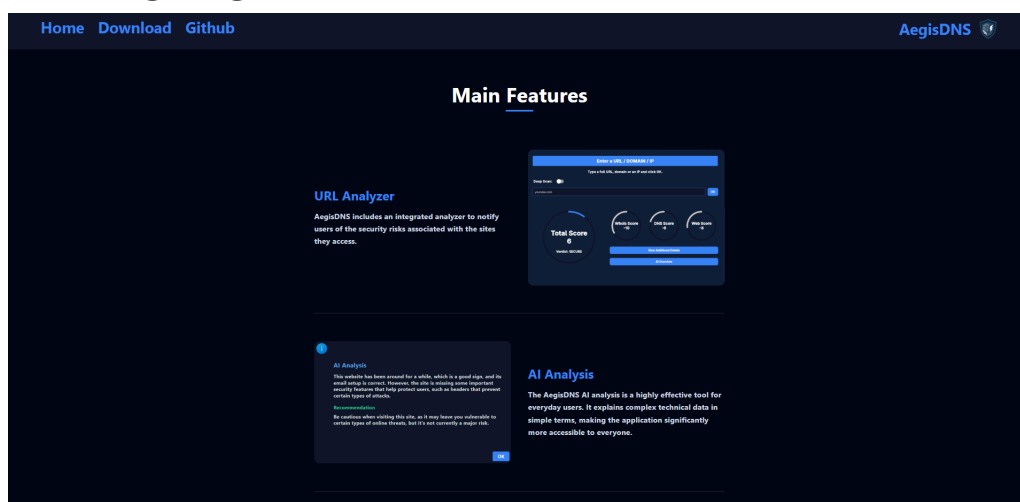


Figure 24: Home section

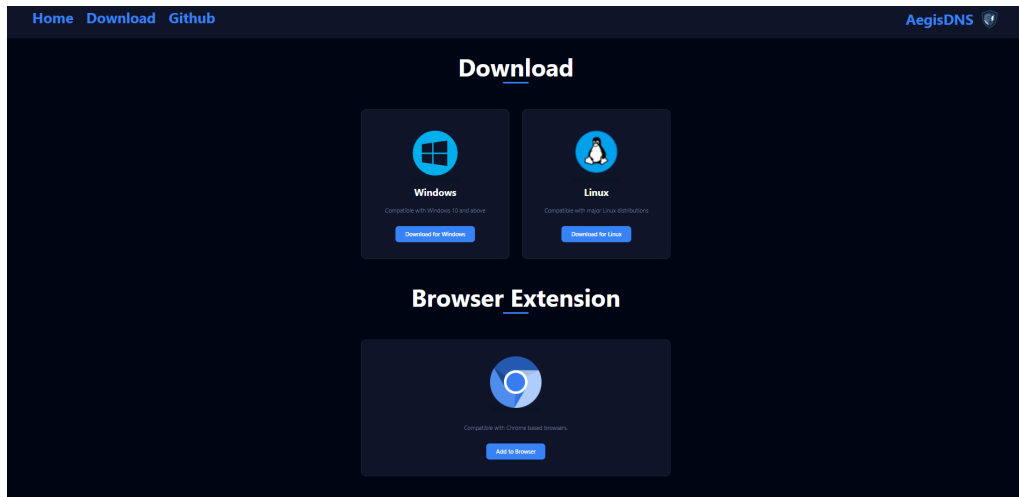


Figure 25: Download section

In order to complement the desktop application, AegisDNS includes a dedicated landing page that serves as the primary public presentation of the project. The landing page is hosted on GitHub Pages and is structured into two main sections. The home section, presented in **Figure 24**, introduces Aegis DNS to possible users, presenting an overview of the system's purpose and highlighting its core features in an accessible format aimed at users regardless of their technical background.

The download section, that can be observe in **Figure 25**, provides direct access to pre-compiled executables for both Windows and Linux, allowing users to install the desktop application without requiring a configured Python environment or familiarity with the command line. The browser extension is also available for direct installation from the same page, consolidating all distribution into a single accessible entry point. The landing page is built as a static site and deployed through GitHub Pages, ensuring reliable availability without the need for dedicated hosting infrastructure.

9. Conclusion

AegisDNS was designed around the premise that everyday users should be able to understand the threats they face without needing a technical background. The result is a multi-component system with many different tools and operation modes, ranging from just a passive navigation logger and background scanner to an active connection interceptor.

The system brings together a PySide6 desktop application, a Manifest V3 compatible browser extension, a local Flask server, a heuristic scanning engine, and a local large language model, all coordinated and synchronized through a communication layer on localhost. Addresses and domains the user comes across while browsing are evaluated in many distinct areas, including WHOIS, DNS, TLS, and HTTP dimensions, gathering evidence from many different sources to produce a single risk score.

The signal and verdict information is then passed onto the AI layer, which uses an LLM to translate the technical language into a plain-language summary with a concrete recommendation for the user. Inference runs entirely on the local machine.

AegisDNS demonstrates that meaningful browser-level security tooling can be built without relying on cloud infrastructure, subscription threat feeds, or prior security knowledge from the user. The architecture is modular enough that individual components such as the scanner heuristics, the extension modes, and the AI model can all be improved independently as the threat landscape evolves.

Bibliography

- [1] The Qt Company, “PySide6 – Qt for Python.” Accessed: June 05, 2026. [Online]. Available: <https://doc.qt.io/qtforpython-6/>
- [2] Docker Inc., “Docker – Accelerated Container Application Development.” Accessed: June 05, 2026. [Online]. Available: <https://www.docker.com/>
- [3] VirusTotal, “VirusTotal – Free Online Virus, Malware and URL Scanner.” Accessed: June 05, 2026. [Online]. Available: <https://www.virustotal.com/>
- [4] Malwarebytes, “Malwarebytes – Cybersecurity Software and Services.” Accessed: June 05, 2026. [Online]. Available: <https://www.malwarebytes.com/>
- [5] M. Bayer, “SQLAlchemy – The Python SQL Toolkit and Object Relational Mapper.” Accessed: June 05, 2026. [Online]. Available: <https://www.sqlalchemy.org/>
- [6] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in *Proceedings of the USENIX Annual Technical Conference*, 1999. Accessed: June 05, 2026. [Online]. Available: <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>
- [7] P. Biondi, “Scapy – Packet Crafting for Python.” Accessed: June 05, 2026. [Online]. Available: <https://scapy.net/>
- [8] Google, “Welcome to Manifest V3 – Chrome Extensions.” Accessed: June 05, 2026. [Online]. Available: <https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3>
- [9] Spamhaus, “Registrar Reputation Statistics.” Accessed: June 05, 2026. [Online]. Available: <https://www.spamhaus.org/reputation-statistics/registrars/domains/>
- [10] The Hacker News, “CISA and FBI Warn Fast Flux is Powering Resilient Malware, Phishing, and C2 Networks.” Accessed: June 05, 2026. [Online]. Available: <https://thehackernews.com/2025/04/cisa-and-fbi-warn-fast-flux-is-powering.html>
- [11] W3Techs, “Usage Statistics of DNS Server Providers for Websites.” Accessed: June 05, 2026. [Online]. Available: https://w3techs.com/technologies/overview/dns_server
- [12] MxToolbox, “MX Records, SPF, DKIM and DMARC.” Accessed: June 05, 2026. [Online]. Available: <https://mxtoolbox.com/dmarc/details/mx-records-spf-dkim-dmarc>
- [13] Barrion, “Security Headers Guide.” Accessed: June 05, 2026. [Online]. Available: <https://barrion.io/blog/security-headers-guide>

- [14] Meta AI, “Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models.” Accessed: June 05, 2026. [Online]. Available: <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>
- [15] A. Betlen, “llama-cpp-python: Python bindings for llama.cpp.” Accessed: June 05, 2026. [Online]. Available: <https://github.com/abetlen/llama-cpp-python>
- [16] Hugging Face, “Hugging Face – The AI Community Building the Future.” Accessed: June 05, 2026. [Online]. Available: <https://huggingface.co/>